

Instituto Tecnológico de Costa Rica
Escuela Ingeniería en Computadores
Curso: CE1101 Introducción a la
programación

Documentación técnica Crazy Snack Rush TEC

Estudiantes:

Felipe Sanabria Pacheco

Kevin Sanabria Quesada

Profesor:

Leonardo Araya Martínez

Semestre I 2026

0. índice

Contenido

0. índice2

1. Introducción.....	3
1.1. Propósito del documento	3
1.2. Descripción del proyecto	3
1.3. Objetivos y Alcance.....	3
2. Descripción General del Sistema	4
2.1. Perspectiva del Producto	4
2.2. Funcionalidades principales	4
2.3. Restricciones del sistema, Supuestos y Dependencias	4
3. Requisitos del Sistema.....	6
3.1. Requisitos Funcionales	6
3.2. Requisitos No Funcionales	6
4. Diseño de la Solución.....	7
4.1. Visión General de la Arquitectura.....	7
4.2. Descripción de Componentes	7
4.4. Diagrama de Clases y Modelo de Datos	8
4.5. Diagrama de Flujo y Secuencia	9
5. Implementación y Validación de Calidad.....	10
5.1. Estructura del repositorio	10
5.2. Entorno de Desarrollo, Guía de instalación y Ejecución.....	10
5.3. Estrategias y Casos de Prueba	12
5.4. Resultados de pruebas y problemas encontrados	12
6.1. Herramientas de IA utilizadas	14
6.3. Declaración de Responsabilidad y Autoría	15
7. Conclusión.....	16
8. Bibliografía	17

1. Introducción

Este proyecto se inspira en el juego Overcooked 2. Se desarrolla un sistema que permite a dos jugadores jugar de manera simultánea una versión simplificada del juego

1.1. Propósito del documento

Este documento describe el diseño, implementación y funcionamiento del sistema Crazy Snack Rush TEC dirigido a estudiantes y profesores

1.2. Descripción del proyecto

El sistema consiste en una versión simplificada del famoso videojuego Overcooked 2, en la que dos personas pueden jugar simultáneamente de manera cooperativa para preparar y entregar recetas. Los jugadores deben coordinar sus acciones para recolectar ingredientes, transportarlos a las estaciones de preparación, cocinarlos cuando sea necesario y finalmente servir los platos terminados a los clientes y así completar y avanzar al siguiente nivel.

1.3 Objetivos y Alcance

Objetivo general:

Desarrollar una versión simplificada de Overcooked 2

Objetivo específico:

Implementar Programación Orientada a Objetos (POO), para el desarrollo del programa

2. Descripción General del Sistema

El sistema permite a dos personas jugar de manera simultánea y colaborativa desde una misma computadora, para lograr un objetivo en específico completar la mayor cantidad de recetas

2.1. Perspectiva del Producto

El sistema opera en Python y está diseñado para trabajar de forma autónoma, por lo que no requiere conexión a servicios externos ni dependencias de otros sistemas para ejecutar sus funciones principales. Toda la lógica del juego, incluyendo la gestión de jugadores, recetas, ingredientes y niveles, se procesa internamente. Esto permite que el juego funcione de manera independiente

2.2. Funcionalidades principales

- [jugabilidad colaborativa]: [los 2 jugadores deben de colaborar para poder realizar y entregar la mayor cantidad de recetas]
- [horno]: [cocina los alimentos que lo requieran para completar la receta y si se deja mucho tiempo en el mismo los alimentos se queman]
- [cortar]: [algunos alimentos deberán ser cortados para cumplir con éxito la receta]
- [traslado de alimentos]: [para poder completar la receta se deben de llevar los alimentos a diferentes puestos para completar su preparación]

2.3. Restricciones del sistema, Supuestos y Dependencias

Limitaciones

- [Pygame]: se debe de contar con la versión 3.13 para poder utilizar Pygame y por ende ejecutar el juego).

Supuestos:

- [teclado]: Los usuarios finales cuentan con un teclado funcional que les permite jugar el juego
- [jugabilidad]: Se espera que el usuario final sepa jugar o una noción de cómo se juega

Dependencias:

- El sistema no presenta ninguna dependencia

3. Requisitos del Sistema

3.1. Requisitos Funcionales

ID	Descripción	Prioridad	Criterio de Aceptación
RF-001	[reconocer la receta entregada]	Alta	[El sistema debe de reconocer efectivamente cuál de las recetas que solicito fue la que se le entrego]
RF-002	[Proporción de puntos]	alta	[El sistema debe de asignar una cantidad de puntos predefinida en base a la calidad del producto entregado]
RF-003	[Asignación de rectas]	media	[el sistema de asignar recetas constantemente para que los jugadores no tengan nunca tiempos muertos]

3.2. Requisitos No Funcionales

ID	Categoría	Descripción	Métrica / Valor Esperado
RNF-001	Rendimiento	El sistema debe responder a las solicitudes del usuario en tiempo aceptable.	Tiempo de respuesta < 0.5 segundos en condiciones normales.
RNF-002	Usabilidad	La interfaz debe ser intuitiva para usuarios no técnicos.	El 60% de los usuarios completan las tareas principales sin ayuda.

4. Diseño de la Solución

4.1. Visión General de la Arquitectura

El sistema utiliza una arquitectura simple basada en Python y en los principios de la Programación Orientada a Objetos (POO). Los elementos del juego se representan mediante clases y objetos que encapsulan sus atributos y comportamientos. Esta estructura facilita la organización del código, mejora su reutilización y permite ampliar funcionalidades del juego de forma sencilla y mantenible.

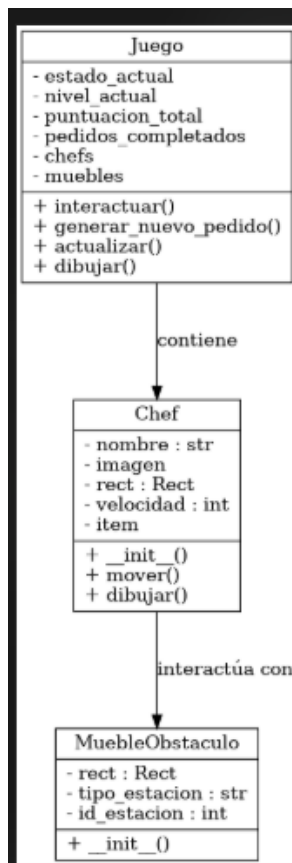
4.2. Descripción de Componentes

Componente	Responsabilidad	Tecnología	Dependencias
Pygame	Interfaz de usuario y experiencia visual.	conjunto de módulos del lenguaje Python	Versión 3.13 de Python
Sistema de puntos	Asignación de puntos por receta entregada	Python (Programación Orientada a Objetos)	Clase pedido
Gestor de niveles	Se encarga de colocar la cocina de cada nivel	Python	Python

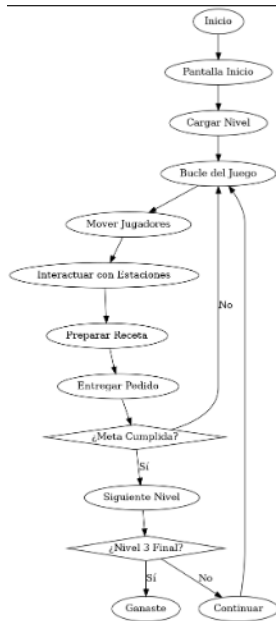
4.3. Stack Tecnológico

Categoría	Tecnología	Versión	Propósito
Lenguaje de programación	[Python]	[3.13]	Desarrollo de código fuente
Librería de juegos	[Pygame]	[2.6.1]	Control general del juego
Modulo estándar-sistema de archivos	[os]	[Python 3.13]	Rutas de archivo y carga de recursos
Modulo estándar-sistema	[sys]	[Python 3.13]	Parámetros de ejecución
Modulo estándar - aleatoriedad	[random]	[Python 3.13]	Generación de eventos aleatorios

4.4. Diagrama de Clases y Modelo de Datos



4.5. Diagrama de Flujo y Secuencia



5. Implementación y Validación de Calidad

5.1. Estructura del repositorio

El proyecto está organizado en una serie de carpetas y archivos que permiten mantener separados los distintos recursos y componentes de la aplicación, facilitando su desarrollo, mantenimiento y escalabilidad.

La carpeta `.vscode` contiene la configuración específica del entorno de desarrollo Visual Studio Code, incluyendo ajustes del editor y preferencias del proyecto.

La carpeta `Alimentos` almacena las imágenes y recursos gráficos correspondientes a los ingredientes utilizados en el juego, como panes, carnes, vegetales, quesos y demás alimentos que intervienen en las mecánicas de preparación.

La carpeta `Escenarios` reúne los fondos y elementos visuales que representan los distintos ambientes o niveles donde se desarrolla el juego.

La carpeta `Extras` contiene recursos complementarios que apoyan el funcionamiento del proyecto, como imágenes adicionales, iconos u otros archivos auxiliares.

La carpeta `Instrumentos` incluye los sprites y recursos gráficos de los utensilios y estaciones de trabajo empleados durante la jugabilidad, tales como cuchillos, cafeteras, basureros y otros objetos interactivos.

La carpeta `Pantalla de Inicio` almacena los elementos gráficos utilizados para el menú principal y la interfaz de bienvenida de la aplicación.

La carpeta `Personajes` contiene los sprites, animaciones y recursos visuales asociados a los personajes presentes en el juego.

Por su parte, los archivos `prueba_1.py`, `prueba_2.py`, `prueba_3.py` y `prueba_4.py` corresponden a módulos desarrollados en Python que implementan la lógica y las funcionalidades del proyecto, organizando el código fuente en diferentes componentes para facilitar su mantenimiento y reutilización.

5.2. Entorno de Desarrollo, Guía de instalación y Ejecución

- Entorno:
 - Sistema Operativo: [Windows 10+]
 - Lenguaje / Runtime: [Python 3.13]

- Guía de instalación y ejecución:

Para ejecutar correctamente el proyecto, es necesario contar con un entorno de desarrollo compatible con Python 3.13 y las bibliotecas requeridas por la aplicación.

En primer lugar, debe instalarse Python 3.13 desde el sitio oficial de Python. Durante el proceso de instalación en Windows, se recomienda habilitar la opción “Add Python to PATH”, lo que permitirá ejecutar Python desde cualquier terminal del sistema sin configuraciones. Si la instalación fue exitosa, el sistema mostrará la versión instalada de Python.

Posteriormente, deben instalarse las dependencias necesarias para el funcionamiento del proyecto

utilizando el administrador de paquetes pip, incluido con Python. Para ello, basta con ejecutar el comando `pip install Pygame` en una terminal. Para bibliotecas secundarias, estas pueden instalarse de la misma manera o mediante un archivo `requirements.txt` utilizando el comando `pip install -r requirements.txt`.

Una vez configurado el entorno, el proyecto puede abrirse en un entorno de desarrollo como Visual Studio Code o ejecutarse directamente desde una terminal ubicada en la carpeta raíz del repositorio. Finalmente, la aplicación se inicia ejecutando el archivo principal del proyecto mediante el comando `python nombre_del_archivo_principal.py`, sustituyendo el nombre por el correspondiente archivo de inicio de la aplicación.

Con estos pasos, el entorno de desarrollo quedará correctamente configurado y el proyecto estará listo para su ejecución y futuras modificaciones.

5.3. Estrategias y Casos de Prueba

Describe la estrategia general de pruebas adoptada: tipos de prueba, herramientas utilizadas, cobertura, objetivo y responsables. Adicionalmente, se definen y documentan los casos de prueba más representativos con identificador, precondiciones, pasos, resultado esperado y resultado obtenido.

Ejemplo:

- Estrategias de pruebas:

Tipo de Prueba	Descripción	Herramienta	Cobertura Objetivo
Pruebas Unitarias	Creación de interfaz de usuario	[VScode/python]	20% del Código
Pruebas Unitarias	Verificación de movimiento de personajes	[VScode/python]	40% del código
Pruebas Unitarias	Verificación de preparación y entrega de recetas	[VScode/python]	70% del código
Pruebas Unitarias	comprobación de requisitos y jugabilidad	[VScode/python]	100% Código

- Casos de prueba:

ID	Descripción	Pasos	Resultado Esperado	Resultado Obtenido	Estado
CP-001	[prueba interfaz]	1. Ejecutar Código 2. Presionar empezar	Se proyecta la interfaz de usuario y avanzo a la siguiente pantalla luego presionar empezar	[fue el mismo que el esperado]	✓ Pasó
CP-002	[prueba imágenes]	1. ejecutar Código 2. presionar "empezar"	Luego de presionar empezar se presenta el mapa	[El mismo que el resultado esperado]	✓ Pasó
CP-003	[jugabilidad]	1. ejecutar Código 2. presionar "empezar"	Luego de presionar empezar se logra una jugabilidad decente	[El resultado fue el esperado]	✓ Pasó

5.4. Resultados de pruebas y problemas encontrados

- Resultados:

0 Casos de prueba exitosos:

- 80% de las pruebas unitarias funcionaron
- Tiempo de ejecución-perfilado
- Imágenes del programa corriendo

Problemas encontrados:

- Problemas Solucionados:
 1. Los personajes pasan a través de los muebles que no se supusiese que atravesaran la solución fue delimitar la zona con bloqueas
 2. Las imágenes no se colocaban y el fondo era blanco, para solucionarlo se revisó y se corrigió la ruta de búsqueda del archivo

6. Declaración de uso de Inteligencia Artificial y herramientas

Principio de Transparencia Académica

El uso de herramientas de Inteligencia Artificial en proyectos académicos requiere declaración explícita y responsable. Esta sección documenta de forma honesta cómo, cuándo y para qué se utilizaron herramientas de IA en el desarrollo de este proyecto, siguiendo los principios de integridad académica

6.1. Herramientas de IA utilizadas

Lista todas las herramientas de IA utilizadas durante el desarrollo. Sé específico con el nombre, versión (si aplica) y propósito.

Ejemplo:

Herramienta de IA	Tipo	Período de Uso	Propósito
[Ej. ChatGPT / GPT-4]	Modelo de lenguaje (LLM)	[junio/2026]	Generación de imágenes como: mapas y personajes además de aportes al código y resolución de dudas técnicas

6.2. Descripción del Uso de Inteligencia Artificial

- Generación y asistencia en código
- Diseño de mapas y personajes

6.3. Declaración de Responsabilidad y Autoría

Esta declaración debe ser redactada y firmada por todos los integrantes del equipo. Afirma que el trabajo final es producto del esfuerzo intelectual del equipo y que la IA fue utilizada como herramienta de apoyo.

Ejemplo:

DECLARACIÓN DE AUTORÍA Y RESPONSABILIDAD

Los suscritos, integrantes del equipo de trabajo del proyecto "[Crazy Snack Rush TEC]", declaramos que:

1. El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de nuestro trabajo intelectual y esfuerzo como equipo.
2. Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.
3. Todo el contenido generado por IA fue revisado, comprendido, validado y, cuando fue necesario, corregido por los integrantes del equipo.
4. Asumimos plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.
5. El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

7. Conclusión

El desarrollo del proyecto permitió cumplir satisfactoriamente los objetivos planteados al inicio, logrando implementar una versión simplificada del juego Overcooked 2 utilizando Python y Programación Orientada a Objetos. Se consiguió crear un sistema funcional que permite la interacción simultánea de dos jugadores, la preparación de recetas y la progresión entre niveles, demostrando la correcta aplicación de los conceptos estudiados durante el curso.

Uno de los principales retos superados fue la coordinación de múltiples elementos dentro del juego, como el movimiento de los jugadores, la manipulación de ingredientes, la gestión de recetas y la detección de eventos. Además, fue necesario diseñar una estructura de clases adecuada que facilitara la organización del código y permitiera mantener el proyecto de manera eficiente conforme se añadían nuevas funcionalidades.

Como posibles mejoras futuras, se podrían incorporar nuevos tipos de recetas, niveles más complejos, efectos visuales y sonoros más avanzados, así como un sistema de guardado de progreso. Estas mejoras permitirían ofrecer una experiencia más cercana al juego original y ampliar las funcionalidades del proyecto.

8. Bibliografía

- [1] Pinterest. (s. f.). Pinterest. Pinterest. <https://es.pinterest.com/>
- [2] Pygame. (s. f.). A new user's guide to pygame. Pygame.
<https://www.pygame.org/docs/tut/newbieguide.html>
- [3] Real Python. (s. f.). Pygame: A primer on game programming in Python. Real Python.
<https://realpython.com/pygame-a-primer/>
- [4] Rana, S. (2024, 12 de marzo). How to see and clean invisible characters in Python. Towards Dev.
<https://towardsdev.com/how-to-see-and-clean-invisible-characters-in-python-25c17fb8dc4e>